

Programming Style

CMPT 145

Copyright Notice

©2020 Michael C. Horsch

This document is provided as is for students currently registered in CMPT 145.

All rights reserved. This document shall not be posted to any website for any purpose without the express consent of the author.

Learning Objectives

After studying this chapter, a student should be able to:

- To be aware of stylistic concerns in programming, such as the importance of good variable names, program documentation, and clarity of code.

Programming: Style counts

Programs must be written for people to read, and only incidentally for machines to execute.

— Harold Abelson

- For beginners, style is less important than concepts.
- Good style is important for bigger programs.
- A well-written program is easier to debug.
- Most software developers build on software that was written by someone else.
- After six weeks, your own programs will feel like someone else wrote it.

Developing good style

- Don't just write code, read code.
- Learn more about Python.
- Browse `Python.org` and Stack Overflow.
- Learn the basics of any other language.

Improving your code

Turn long if-statements into lookups

- Bad:

```
1  if month == 1:  
2      print('Jan')  
3  elif month == 2:  
4      print('Feb')  
5  # etc, 12 months
```

- Good:

```
1  month_names = ['Jan', 'Feb', 'Mar', # and the rest  
2  print(month_names[month-1])
```

Improving your code

Turn long if-statements into lookups

- Bad:

```
1  if month_name == 'Jan':  
2      month = 1  
3  elif month_name == 'Feb':  
4      month = 2  
5  # etc, 12 months
```

- Good:

```
1  month_dict = {'Jan':1, 'Feb':2, 'Mar':3, # and the rest  
2  month = month_dict[month_name]
```

Improving your code

Factor common statements out.

- Poor:

```
1  if condition:
2      statement A
3      statement B
4  else:
5      statement A
6      statement C
```

- Better:

```
1  statement A
2  if condition:
3      statement B
4  else:
5      statement C
```


Improving your code

Factor common statements out.

- Poor:

```
1  if condition:
2      statement A
3      statement B
4  else:
5      statement C
6      statement B
```

- Better:

```
1  if condition:
2      statement A
3  else:
4      statement C
5  statement B
```

Improving your code

Factor common statements out.

- Poor:

```
1 for v in range(some):  
2     statement A # doesn't depend on v  
3     statement B # depends on v
```

- Better:

```
1 statement A  
2 for v in range(some):  
3     statement B # depends on v
```

Improving your code

Don't check for end-of-loop inside your loops!

- Poor:

```
1 i = 0
2 while i < N:
3     statement B
4     if i == N - 1:
5         statement A
6     i += 1
```

- Better:

```
1 i = 0
2 while i < N:
3     statement B
4     i += 1
5 statement A
```

Improving your code

Don't use if-statements when a Boolean expression will do.

- Poor:

```
1 def function(a,p):  
2     if a == 3 or p < 0.005:  
3         return True  
4     else:  
5         return False
```

- Better:

```
1 def function(a,p):  
2     return a == 3 or p < 0.005
```

Improving your code

Don't use a while loop for a list, even if you need indices!

- Poor:

```
1 i = 0
2 while i < len(some_list):
3     statements about some_list[i]
4     i += 1
```

- Better:

```
1 for ind, val in enumerate(some_list):
2     statements about some_list[ind]
3     statements about val
```

- `enumerate(alist)` returns a sequence of tuples (i, v) , where i is the index of the value v in the given list `alist`.

Style Guidelines

Write clearly – don't be too clever

- Poor:

```
1 new_cl = [[1 for l in c if (-1)*flip*first_lit != 1]
2           for c in clauses if flip*first_lit not in c]
```

- Good:

```
1 new_clauses = []
2 for clause in clauses:
3     if flip * first_lit not in clause:
4         new_clause = []
5         for literal in clause:
6             if (-1) * flip * first_lit != literal:
7                 new_clause.append(literal)
8         new_clauses.append(new_clause)
```

Style Guidelines

Break complex expressions into smaller pieces.

- Poor:

```
1 new_cl = [[1 for l in c if (-1)*flip*first_lit != 1]
2           for c in clauses if flip*first_lit not in c]
```

- Good:

```
1 new_clauses = []
2 for clause in clauses:
3     if flip * first_lit not in clause:
4         new_clause = []
5         for literal in clause:
6             if (-1) * flip * first_lit != literal:
7                 new_clause.append(literal)
8         new_clauses.append(new_clause)
```

Style Guidelines

Replace **repetitive expressions** by calls to a **common function**

- Poor:

```
1 a = 10
2 b = 12
3 fa = 1
4 for i in range(1,a):
5     fa = fa * i
6 fb = 1
7 for i in range(1,b):
8     fb = fb * i
9 fab = 1
10 for i in range(1,a+b-1):
11     fab = fa * i
12 print('Mario paths:', fab/(fa*fb))
```

- Good:

```
1 def fact(n): # ... assume standard definition
2 a = 10
3 b = 12
4 print('Mario paths:', fact(a+b-2)/(fact(a-1)*fact(b-1)))
```


Style Guidelines

Choose good variable names

- Good names **remind** the reader of the purpose
- Bad names require the reader to **deduce** the purpose
- Good:

```
1 clauses = # ...  
2 left_subtree = # ...  
3 number_of_elements = # ...
```

- Bad:

```
1 cs = # ...  
2 l = # ...  
3 n = # ...
```

Style Guidelines

Use short names for loop control variables

- Good:

```
1  for v in alist:  
2      # do stuff with v
```

- Bad:

```
1  for value_found_in_alist in alist:  
2      # do stuff with value_found_in_alist
```

Style Guidelines

Avoid using variable names **l** (ELL) and **o** (OH)

- Too similar to **1** and **O**
- Bad:

```
1 for l in clauses:  
2     # do stuff with l
```

- Better:

```
1 for cl in clauses:  
2     # do stuff with cl
```

Style Guidelines

Don't over-comment

- Comments should be about what's **not obvious**.
- Bad:

```
1 i = 0 # list index for summing the list
2 total = 0 # running total for summing the list
3 while i < len(some_list):
4     total = total + i # add i to the running total
5     i += 1 # increment i
```

- Better:

```
1 # calculate the total cost from the list of costs
2 i = 0
3 total = 0
4 while i < len(some_list):
5     total = total + i
6     i += 1
```

Advice without examples

Don't **patch** bad code – **rewrite** it

- Sometimes there's no way to **nudge** a program to make it better
- Treat your first implementation as a prototype
- Prototypes get **replaced completely** all the time
- Learn when **you personally** need to delete and start again from scratch.

Advice without examples

Making your program faster.

- Make it right **before** you make it faster
- **Keep it right** when you make it faster
- Make it clear before you make it faster
- Don't make **trivial** "optimizations" to make code trivially faster. Think about your algorithm instead!
- Make sure your function is **important enough** to make faster.

How to be more a more efficient programmer

- Learn to touch type.
- Read as much code as you write.
- Always practice coding. Write little programs every day.
- Keep a journal of the errors that caused you grief.
- Learn more about your language. Not just what gets covered in class.
- Browse Python.org and Stack Overflow. Read. Learn.
- Learn to use the IDE properly.
- Stop using the mouse and menu. Use and memorize key-commands.